

Profiling and Optimizing the TSFM MCP Server

As part of the IBM AssetOpsBench project, our team developed a reproducible benchmarking harness, stage-level profiling system, and interchangeable model interface that identified preprocessing and inference bottlenecks, achieving up to **12.8 × faster forecasting** and **12.2% lower fine-tuning latency** while supporting forecasting, fine-tuning, and anomaly detection workflows.

Team #19 (Lion Latency)

Sally Go · yg3066

Sam Colman · sc5750

Byeolah Kwon · bk2833

Tomas Pasiecznik · tp2758

GitHub: <https://github.com/Lion-Latency/AssetOpsBench>

WandB: <https://wandb.ai/lion-latency/hpml-project-final>

Motivation & Problem

Time budget: 1 minute.

Why this matters

- **Real-world setting where this workload runs:** This workload runs in high-frequency industrial telemetry environments, where AI agents monitor physical assets such as chillers and turbines and use MCP tools for tasks such as forecasting and anomaly detection.
- **Cost / latency / energy pressure:** The large number of sensors in these environments require systems with low latency and minimal compute costs. Optimized workloads are critical for creating safe and viable real-time AI systems. For example, delays in anomaly detection can be serious when a dangerous malfunction is occurring.
- **Why a naive baseline is not good enough:** A naive baseline cannot reveal which stage of the pipeline is causing a bottleneck. Stage-level benchmarking provides the level of granularity needed to identify bottlenecks and apply meaningful, targeted optimizations.

Problem statement

We aim to optimize the TSFM MCP Server, which is tightly coupled with IBM's Granite TinyTimeMixer (TTM) as its model, by profiling stage-level latency bottlenecks across forecasting, fine-tuning, anomaly detection, and integrated anomaly detection workloads including data retrieval, preprocessing, model loading, inference/training, and serialization, while introducing a method of decoupling the server from TTM through an interchangeable model interface.

Goal: 4x total speedup at iso-accuracy for persistent servers (not Plan-Execute)

Build external harness allowing for profiling of TSFM server with non-TTM time series models.

Technical Approach — Model & Data

Model

Primary model:

IBM TinyTimeMixer, [ttm_96_28](#)

Comparison model:

Chronos / [chronos-2](#)

Workflows:

forecasting, fine-tuning, TSAD, integrated TSAD

Dataset

Source:

IBM AssetOpsBench real industrial time-series data

Benchmark sample:

100K rows real-data sample

Asset-Level Evaluation: 5 Valid

Equipment Groups

CQPA AHU 2B, Chiller 3, Chiller 4, Chiller 6, Chiller 9

Hardware

Cloud platform: Google Cloud Deep Learning VM

Accelerator: 1× NVIDIA L4 GPU, 24 GB VRAM

CPU: 4 vCPUs / 8 logical CPUs for data loading, preprocessing, orchestration

Runtime stack: Python 3.12, PyTorch, Hugging Face Transformers, CUDA

Tracking: Weights & Biases

Technical Approach — Optimizations Applied

1. Preprocessing cache

Avoids repeated data quality filtering and preprocessing when the same sampled data is reused.

2. Preprocessing parallelism

Parallelizes preprocessing work across asset groups to reduce CPU-side bottlenecks.

3. Cache + Parallelism combined preprocessing

Combines preprocessing caching and parallel execution to minimize redundant work and reduce CPU-side bottlenecks

Profiling Methodology

Tools

Benchmark driver: `benchmark_runner.py`

Instrumentation: internal stage-level timing hooks

Memory tracking: CPU/RSS and GPU memory deltas

Experiment tracking: Weights & Biases, `hpml-project-final`

Metrics Captured

Latency: end-to-end runtime per workflow

Stage timing: data retrieval, quality filter, preprocessing, model loading, inference/training, serialization

Overhead: non-model execution time

Memory: CPU/RSS delta and GPU memory delta

Reliability: success/failure status per asset group

Quality metrics: forecasting / fine-tuning performance metrics when available

Results — Headline Numbers

12.8x

Faster forecasting

[Chronos: 3.68s vs TTM: 46.95s]

12.2%

Lower fine-tuning latency

[TTM baseline: 12.89s → parallelism_only: 11.32s]

51min

Largest bottleneck

[Integrated TSAD, TTM Combined, Chiller 3 only: 3076.8s]

*Forecasting/fine-tuning results are averaged across 5 valid asset groups.
TSAD/integrated TSAD results are reported on Chiller 3 only due data and memory constraints.*

Results — TTM Optimization Impact

TTM: Baseline vs. Best Optimization

Workflow	Baseline (s)	Best Observed (s)	Best Mode	Change vs. Baseline
Forecasting	46.95	46.24	combined	1.5% faster
Fine-tuning	12.89	11.32	parallelism_only	12.2% faster
TSAD*	109.79	107.89	cache_only	1.7% faster
Integrated TSAD*	2335.64	2335.64	baseline	No improvement

Takeaway

- Optimization impact was workflow-dependent.
- Fine-tuning benefited most, with parallelism reducing latency by 12.2%.
- Forecasting and TSAD showed only marginal gains.
- Integrated TSAD remained the largest bottleneck and became slower under cache/parallelism.

Results — TTM vs. Chronos

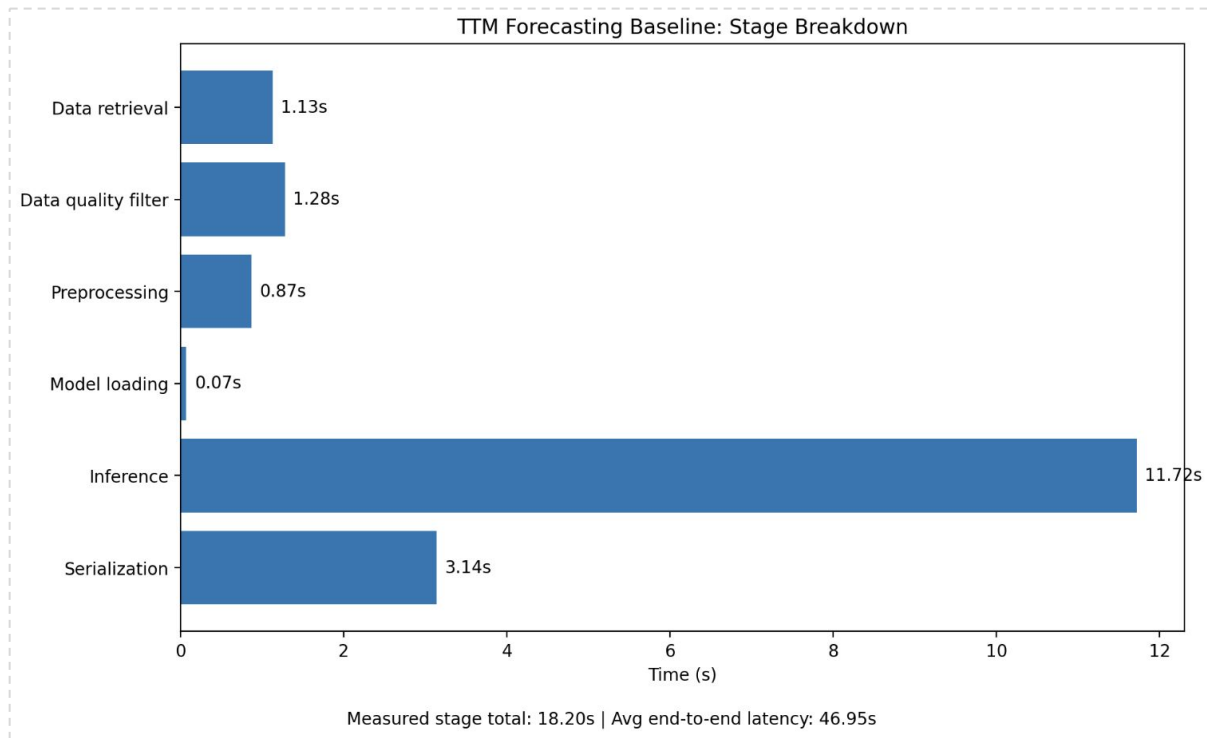
Same 100K benchmark harness, different model backend. TSAD results are Chiller 3 only.

Workflow	TTM baseline	Chronos baseline	Result
Forecasting	46.95s	3.68s	Chronos 12.8× faster
Fine-tuning	12.89s	1048.67s	Chronos 81.3× slower
TSAD*	109.79s	98.03s	Chronos slightly faster, but not directly comparable
Integrated TSAD*	2335.64s	133.77s	Chronos faster, but not directly comparable

Takeaway

- Chronos is highly efficient for zero-shot forecasting.
- TTM is much faster for fine-tuning.
- Anomaly detection exposed backend-specific preprocessing/windowing differences.
- We separate model-backend comparison from pipeline optimization results.

Stage-Level Profiling — Forecasting Breakdown

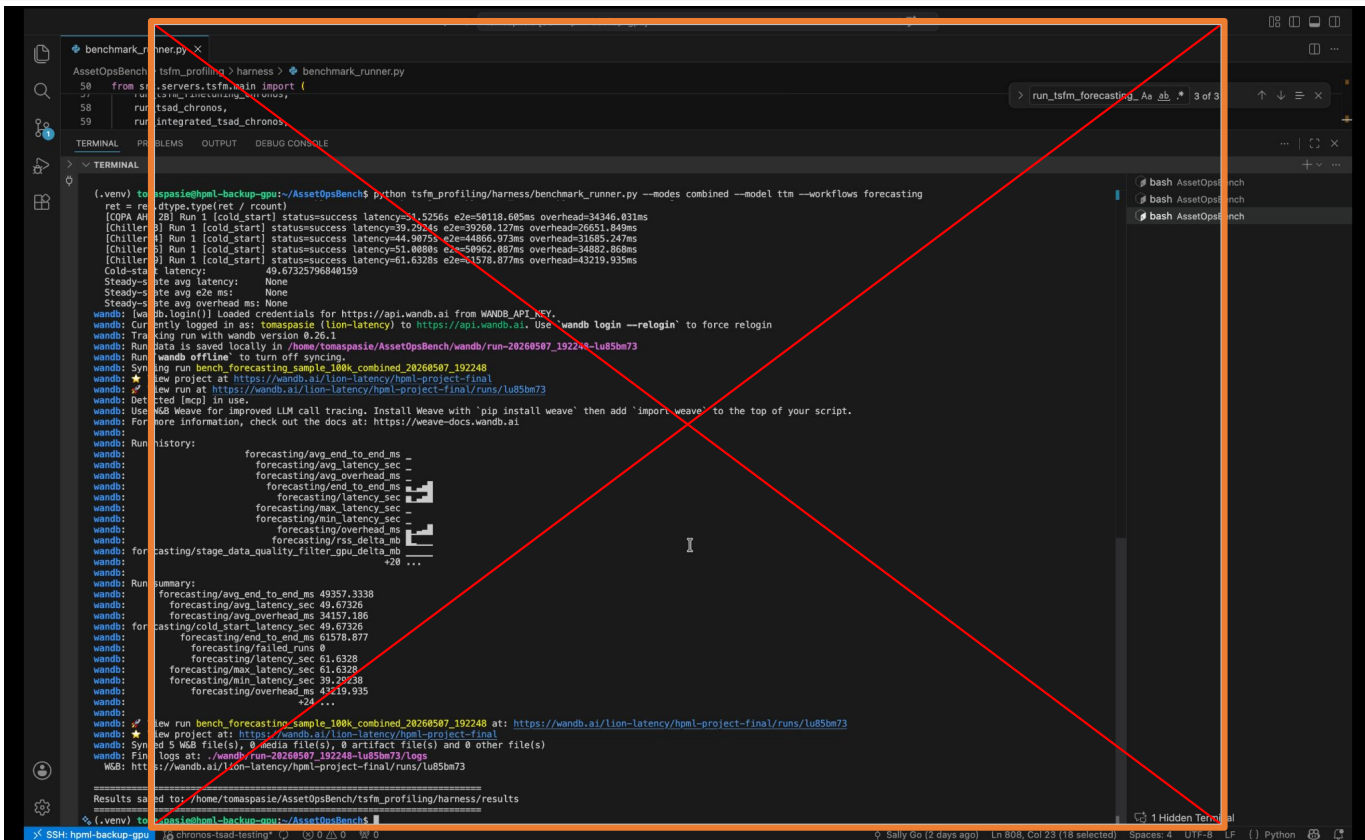


What to point at

- Inference dominates measured stage time.
- Preprocessing is small, explaining why cache/parallelism gave only marginal forecasting gains.
- Model loading is negligible for TTM forecasting.
- End-to-end latency exceeds measured stage total, suggesting additional MCP/tool orchestration overhead.

Demo / Visualization

<https://wandb.ai/lion-latency/hpml-project-final>



```
AssetOpsBench tsfm_profiling > harness > benchmark_runner.py
58 from servers.tsfm.main import (
59     run_tsfm_chronos,
60     run_tsfm_integrated_chronos,
61 )
62
63 if __name__ == '__main__':
64     run_tsfm_chronos()
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

```
(.venv) tomaspie@hpml-backup-gpu:~/AssetOpsBench$ python tsfm_profiling/harness/benchmark_runner.py --modes combined --model tts --workflows forecasting
ret = re.dtype.type(ret / rcount)
[CPA API] Run 1 [cold_start] status=success latency=31.5256s e2e=58118.685ms overhead=34346.831ms
[Chiller] Run 1 [cold_start] status=success latency=39.2924s e2e=39268.127ms overhead=26651.849ms
[Chiller] Run 1 [cold_start] status=success latency=44.8875s e2e=44866.97ms overhead=31685.247ms
[Chiller] Run 1 [cold_start] status=success latency=51.8888s e2e=50962.887ms overhead=34882.868ms
[Chiller] Run 1 [cold_start] status=success latency=61.6328s e2e=61578.877ms overhead=43219.935ms
Cold-start latency: 49.67325796840159
Steady-state avg latency: None
Steady-state avg e2e ms: None
Steady-state avg overhead ms: None
wandb: [wandb.login()] Loaded credentials for https://api.wandb.ai from WANDB_API_KEY.
wandb: Currently logged in as: tomaspie (lion-latency) to https://api.wandb.ai. Use 'wandb login --relogin' to force relogin
wandb: Tracking run with wandb version 0.26.1
wandb: Run data is saved locally in /home/tomaspie/AssetOpsBench/wandb/run-20260507_192248-lu85bm73
wandb: Run 'wandb offline' to turn off syncing.
wandb: Syncing run bench_forecasting_sample_100k_combined_20260507_192248
wandb: ⭐ New project at https://wandb.ai/lion-latency/hpml-project-final
wandb: ⭐ New run at https://wandb.ai/lion-latency/hpml-project-final/runs/lu85bm73
wandb: ⚠️ Weave for improved LLM call tracing. Install Weave with 'pip install weave' then add 'import weave' to the top of your script.
wandb: For more information, check out the docs at: https://weave-docs.wandb.ai
wandb: Run history:
wandb: forecasting/avg_end_to_end_ms -
wandb: forecasting/avg_latency_sec -
wandb: forecasting/avg_overhead_ms -
wandb: forecasting/end_to_end_ms -
wandb: forecasting/latency_sec -
wandb: forecasting/max_latency_sec -
wandb: forecasting/min_latency_sec -
wandb: forecasting/overhead_ms -
wandb: forecasting/rss_delta_mb -
wandb: forecasting/stage_data_quality_filter_gpu_delta_mb -
wandb: forecasting/avg_end_to_end_ms 49357.3338
wandb: forecasting/avg_latency_sec 49.67326
wandb: forecasting/avg_overhead_ms 34157.186
wandb: forecasting/cold_start_latency_sec 49.67326
wandb: forecasting/end_to_end_ms 61578.877
wandb: forecasting/failed_runs 0
wandb: forecasting/latency_sec 61.6328
wandb: forecasting/max_latency_sec 61.6328
wandb: forecasting/min_latency_sec 39.29238
wandb: forecasting/overhead_ms 43219.935
wandb: forecasting/rss_delta_mb 0
wandb: forecasting/stage_data_quality_filter_gpu_delta_mb 0
wandb: ⚠️ New run bench_forecasting_sample_100k_combined_20260507_192248 at: https://wandb.ai/lion-latency/hpml-project-final/runs/lu85bm73
wandb: ⚠️ New project at: https://wandb.ai/lion-latency/hpml-project-final
wandb: Synced 5 W&B file(s), 0 media file(s), 0 artifact file(s) and 0 other file(s)
wandb: Find logs at: ./wandb/run-20260507_192248-lu85bm73/logs
wandb: http://wandb.ai/lion-latency/hpml-project-final/runs/lu85bm73
Results saved to: /home/tomaspie/AssetOpsBench/tsfm_profiling/harness/results
(.venv) tomaspie@hpml-backup-gpu:~/AssetOpsBench$
```

Lessons Learned

What worked

- A 100K real-data sample made the benchmark runnable and comparable.
- Stage-level instrumentation separated model time from data/preprocessing overhead.
- Parallelizing forecasting and anomaly detection on different features of the same asset.

What didn't work

- Full 2.29M-row dataset was too large for a realistic single MCP call.
- TSAD failed initially due to target column / shape mismatch issues.
- ttm_512_96 introduced setup ambiguity and higher memory risk, so ttm_96_28 was safer for the main benchmark.

Overarching Lessons

1. Scope is crucial!
2. Real use-case is the key driver.
3. One-size-fits-all optimizations aren't so simple

Future Works

1. Add model caching for repeated MCP tool calls within one agent session.
2. Implementing `torch.compile(mode="reduce-overhead")`.
3. Adjust preprocessing caching to more closely fit real use-cases.
4. Add stable AMP support and assess latency/performance/memory metrics.
5. Further exploration of Amazon Chronos as alternative to TTM
6. Generalization of harness to accept any time series model.

Teamwork & Contributions

Member	Primary contributions	Tools / artifacts
Sally Go (yg3066)	External harness, setting W&B	benchmark_runner.py, W&B hpml-project-final
Sam Colman (sc5750)	Internal instrumentation, preprocessing caching, performance metric logging	Profiling control, cache, initial parallelization script.
Byeolah Kwon (bk2833)	Profiler and baseline execution, preprocessing parallelism	src/servers/tsfm/ main.py , W&B logs
Tomas Pasiecznik (tp2758)	Environment and reproducibility setup, Chronos integration	Documentation with commands to reproduce results.

Thank you

Questions?